BENNETT UNIVERSITY
THE TIMES GROUP

REAL-TIME SIGN LANGUAGE TRANSLATION USING HAND LANDMARK RECOGNITION

A MINI-PROJECT REPORT (CSET312)

BY

Naitik Maurya (S24CSEU1653)

Priyanshu Saini (S24CSEU1567)

Tejaswita Tiwari (S24CSEU0079)

SUBMITTED TO

SCHOOL OF COMPUTER SCIENCE ENGINEERING AND TECHNOLOGY, BENNETT UNIVERSITY

GREATER NOIDA, 201310, UTTAR PRADESH, INDIA

Dec 2026

DECLARATION

I/We hereby declare that the work presented in the report entitled “REAL-TIME SIGN LANGUAGE TRANSLATION USING HAND LANDMARK RECOGNITION” is an original academic project carried out as part of CSET312 at the School of Computer Science Engineering and Technology, Bennett University, Greater Noida.

The report has been prepared from our own design, implementation work and study of the cited literature. Wherever ideas, datasets, software frameworks or research findings from other authors have been used, they have been identified through appropriate references. The work has not been submitted by me/us for the award of any other degree or academic qualification elsewhere.

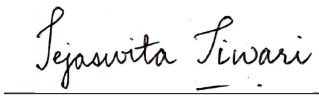
Signature of Candidates



Naitik Maurya
S24CSEU1653



Priyanshu Saini
S24CSEU1567



Tejaswita Tiwari
S24CSEU0079

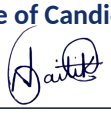
ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project mentor and the faculty members of the School of Computer Science Engineering and Technology, Bennett University, for their guidance throughout this mini-project. Their feedback helped us turn an initial idea into a structured and testable system.

We are also thankful to the university for providing the academic environment and computing resources needed to study computer vision, machine learning and human–computer interaction. The project benefited from publicly available research datasets and open-source software such as MediaPipe, OpenCV and scikit-learn.

Finally, we appreciate the support of our classmates, friends and family members who helped with testing, discussion and feedback. Their practical suggestions were useful when designing the webcam workflow and deciding how the final prototype should present recognition results.

Signature of Candidates



Naitik Maurya
S24CSEU1653



Priyanshu Saini
S24CSEU1567



Tejaswita Tiwari
S24CSEU0079

TABLE OF CONTENTS

LIST OF TABLES ..... **v**

LIST OF FIGURES ..... **vi**

LIST OF ABBREVIATIONS ..... **vii**

ABSTRACT ..... **viii**

1. INTRODUCTION **1**

1.1. Problem Statement 1

1.2. Project Scope 1

2. BACKGROUND RESEARCH **2**

2.1. Goals / Objectives 3

2.2. Literature Review & Project Feasibility 3

3. METHODOLOGY **4**

3.1. Proposed System Overview 4

3.2. Users and Roles 5

3.3. Detailed Workflow 5

3.4. Design Diagrams 5

3.4.1. Use-case Diagram 5

3.4.2. Class Diagram 6

3.4.3. ER / Data Architecture 6

3.5. User Interface 6

3.5.1. User Description and Mockup 6

3.6. System Implementation 7

3.6.1. Algorithms / Pseudo Code 8

3.6.2. Source Code with Documentation 8

4. CONCLUSION **9**

4.1. Goals / Vision 9

4.2. Delivered Solution 9

4.3. Remaining Work 9

5. REFERENCES **10**

LIST OF TABLES

Table 1: Goal and Objectives 3

Table 2: Dataset and Research Resource Comparison 3

Table 3: System Resources and Tools 7

Table 4: User Roles and Responsibilities 5

The tables are numbered according to their first appearance. They summarise the project goals, research resources and implementation decisions without replacing the detailed discussion in the main text.

LIST OF FIGURES

Figure 1: Proposed System Workflow 4
Figure 2: System Architecture 6
Figure 3: Use-case Diagram 5
Figure 4: Activity Diagram 5
Figure 5: Sequence Diagram 5
Figure 6: Data Architecture 6
Figure 7: Dataset Comparison 3

The diagrams are original representations prepared for this project. They describe the intended architecture, workflow and interaction between the major components.

LIST OF ABBREVIATIONS

Abbreviation	Explanation
ASL	American Sign Language
ML	Machine Learning
AI	Artificial Intelligence
RGB	Red, Green, Blue
FPS	Frames Per Second
UI	User Interface
CNN	Convolutional Neural Network
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
NLP	Natural Language Processing
DFD	Data Flow Diagram
UML	Unified Modeling Language
ER	Entity–Relationship
SLR	Sign Language Recognition
SLT	Sign Language Translation
API	Application Programming Interface

ABSTRACT

This mini-project presents a practical prototype for translating American Sign Language (ASL) hand gestures into text and, in the reverse direction, converting text into a sequence of ASL signs. The main design decision is to avoid training a large vision model directly on raw video frames. Instead, a webcam stream is processed with MediaPipe Hand Landmarker to obtain a compact representation of hand geometry. Each detected hand is represented by 21 three-dimensional landmarks, giving 63 numerical coordinates before normalization. For static fingerspelling, these features are supplied to a Random Forest classifier. Dynamic signs are represented as ordered landmark sequences so that the same data structure can later be used with LSTM, GRU or Transformer-based sequence models.

The reverse direction uses a lightweight text-processing stage. English input is converted into a simple ASL-style gloss representation using transparent rules, after which available word-level signs are retrieved from a local sign library. When a whole-word sign is unavailable, fingerspelling can be used as a fallback. This approach is intended as an extensible student prototype rather than a claim of complete linguistic translation.

The project is feasible on an ordinary laptop because landmark extraction substantially reduces the amount of data that must be learned by the classifier. Public datasets such as WLASL, MS-ASL, ASL Citizen and RWTH-PHOENIX-Weather provide useful research references and data resources, while self-recorded webcam samples can be used for controlled testing. The resulting architecture provides a clear path from a small static-sign demonstrator to a broader continuous sign-language system.

Keywords: Sign Language Recognition; ASL; MediaPipe; Hand Landmarks; Random Forest; Computer Vision; Machine Learning

1 INTRODUCTION

Sign language is a complete visual language used by Deaf and hard-of-hearing communities. Unlike spoken communication, it conveys information through coordinated hand shape, movement, location and, in many situations, non-manual cues such as facial expression and body posture. Computer vision therefore offers an attractive way of building communication tools that can interpret visible signing without requiring special gloves or dedicated sensors.

Recent research has shown that sign recognition can be approached at several levels. Some systems classify isolated signs from short videos, while others attempt continuous translation from a complete signing sequence into written or spoken language. Large datasets have made deep learning practical, but these approaches often require substantial computation and carefully labelled video. For a university mini-project, reproducing such a pipeline end-to-end can be difficult.

The proposed work takes a deliberately lightweight route. A normal webcam provides the input, MediaPipe extracts hand landmarks, and a conventional machine-learning classifier operates on the resulting numerical features. This separates hand tracking from sign classification and makes the learning problem smaller, easier to inspect and easier to reproduce. The same landmark representation is retained for dynamic signs, creating a foundation for later sequence modelling.

The project is therefore positioned as a functional prototype with two directions: gesture-to-text recognition and text-to-sign generation. It focuses first on static ASL fingerspelling and then provides a structured extension path for whole-word signs.

1.1. Problem Statement

Current sign-language recognition systems can be difficult to reproduce in a student environment because many rely on large video datasets, deep spatio-temporal networks and substantial training resources. Even when recognition accuracy is strong, the system may cover a restricted vocabulary or a controlled recording setup. The reverse direction—turning ordinary text into natural sign-language output—is also considerably more difficult because sign languages have their own linguistic structure and cannot be treated as a simple word-for-word substitution. The problem addressed by this project is therefore to design a low-cost, understandable and extensible prototype that can recognise selected ASL hand shapes from a webcam and provide a practical route from text back to signs. The system should operate on standard laptop hardware, use a representation that is easy to collect and inspect, and keep the recognition and generation components modular so that the vocabulary can be expanded later.

1.2. Project Scope

- Static ASL fingerspelling is the first recognition target.
- Dynamic word signs are represented as landmark sequences and included as an extension path.
- Text-to-sign generation uses a limited, transparent gloss conversion and a local sign library.
- The project targets a laptop/webcam prototype rather than a certified accessibility product.
- Facial expression, body pose and complete linguistic translation are outside the first prototype scope.

2 BACKGROUND RESEARCH

The literature shows a progression from small, controlled sign-image datasets to large video corpora and continuous translation benchmarks. WLASL is a major word-level ASL resource containing more than 2,000 words and videos from more than 100 signers. Its authors evaluated both appearance-based and pose-based approaches, illustrating the challenge of recognising signs across different people and recording conditions [1]. MS-ASL similarly established a large benchmark with 1,000 signs and more than 200 signers, using ordinary RGB video rather than specialised sensing hardware [2]. These datasets are valuable because they expose models to variation that is absent from small classroom datasets. ASL Citizen follows a community-sourced direction for isolated sign recognition and emphasises the importance of data collected from real contributors [3]. Such resources are useful for research, but they also highlight a practical issue: large video datasets contain considerable visual variation and are not automatically easy to train on a modest computer.

For continuous sign-language translation, RWTH-PHOENIX-Weather and its 2014T extension are important benchmarks. The 2014T corpus combines sign-language video, gloss annotations and German translations, allowing research to move beyond isolated word classification toward sentence-level translation [4]. The benchmark is based on German Sign Language rather than ASL, so it is a useful methodological reference but should not be treated as an ASL dataset.

A second research direction is the use of skeletal or landmark representations instead of raw pixels. MediaPipe Hands demonstrated a real-time pipeline that predicts a hand skeleton from a single RGB camera using a palm detector and a landmark model [5]. For this project, that idea is particularly useful because it allows the webcam image to be converted into a compact geometric representation before classification.

The literature also exposes an important linguistic gap. A classifier can recognise a sign label without understanding the grammar of the target sign language. Research surveys therefore distinguish sign recognition from sign-language translation and sign-language production [6]. The proposed project follows this distinction: the first prototype uses a transparent rule-based gloss conversion stage rather than claiming that a small rule set provides complete ASL grammar. More advanced work can replace this component with a trained language model and richer linguistic annotations.

Overall, existing research motivates a hybrid design. Large video models demonstrate what is possible at scale, while landmark-based processing offers a more manageable starting point for a student prototype. The research gap addressed here is not the claim that no sign-language systems exist; instead, it is the need for a reproducible, low-compute educational system that connects recognition and generation in one pipeline and makes its intermediate representations visible.

2.1. Goals / Objectives

Table 1: Goal and Objectives

#	Goal or Objective
1	Recognise static ASL fingerspelling from webcam frames using hand landmarks.
2	Keep the recognition pipeline lightweight enough for CPU-based development and demonstration.
3	Represent dynamic signs as ordered landmark sequences for future sequence-model training.
4	Convert simple English input into an ASL-oriented gloss representation.
5	Retrieve recorded signs where available and use fingerspelling as a fallback.
6	Design the software so that additional signs, datasets and models can be added without rebuilding the complete system.

2.2. Literature Review & Project Feasibility

The literature supports the feasibility of the proposed architecture because hand-landmark extraction can be performed in real time and the resulting feature vectors are small enough for conventional classifiers. Economic feasibility is favourable for a student prototype because the core software stack is open source and the required hardware is limited to a webcam and a standard laptop. Operational feasibility depends mainly on stable lighting, a visible hand and a sufficiently diverse training set.

Table 2: Dataset and Research Resource Comparison

Resource	Language	Primary Use	Scale / Characteristic
WLASL	ASL	Word-level recognition	>2,000 words; >100 signers
MS-ASL	ASL	Word-level recognition	1,000 signs; >200 signers
ASL Citizen	ASL	Isolated signs	Community-sourced resource
RWTH-PHOENIX-2014T	German SL	Continuous translation	Video + gloss + German text
Self-recorded	ASL	Controlled testing	Owned task-specific samples

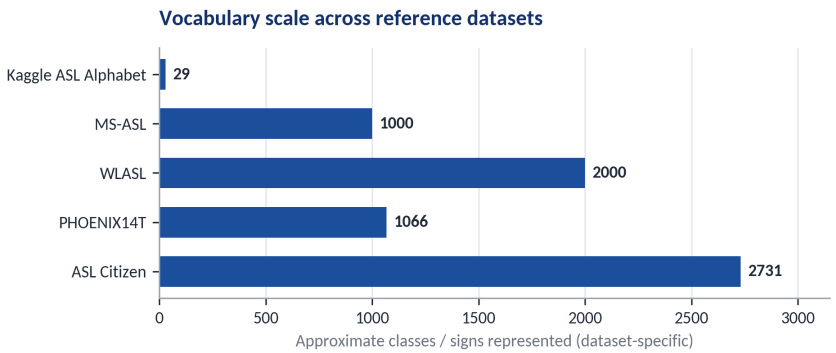


Figure 7: Dataset comparison used to position the proposed prototype.

3 METHODOLOGY

3.1. Proposed System Overview

The proposed system follows a modular pipeline. First, a webcam captures the user's hand. MediaPipe Hand Landmarker detects the hand and returns 21 landmarks per hand. Each landmark contains x, y and z coordinates, producing a 63-value feature vector for a single hand. Before classification, the coordinates can be translated relative to a reference point such as the wrist and scaled using a hand-size measure. This reduces sensitivity to the hand being positioned at different locations in the camera frame. For static signs, the normalised feature vector is passed to a Random Forest classifier. Random Forest is appropriate for an initial prototype because it works well with tabular features, needs comparatively little tuning, trains quickly on a CPU and can provide class probabilities. The predicted class is displayed as a letter or selected sign label. A temporal smoothing step can be added so that a single unstable frame does not immediately change the displayed result.

Dynamic signs are treated differently. Instead of reducing an entire movement to one frame, the system stores a sequence of landmark vectors over time. The resulting representation can be described as frames × 21 × 3. This structure can later be fed to an LSTM, GRU, temporal convolution model or Transformer encoder. The prototype therefore does not force dynamic gestures into a static classifier.

For text-to-sign generation, the input sentence is first normalised and divided into tokens. A small rule-based gloss module can remove articles in selected patterns, handle simple forms of the verb “to be”, and move time expressions toward the beginning of the gloss. The output is deliberately described as an ASL-style gloss approximation, not as a complete grammar engine. Each token is then searched in a local sign library. If a matching word-level sequence is available, it is played or rendered; otherwise the word can be fingerspelled.

Evaluation should be performed using a held-out test set rather than training samples. For static classification, accuracy, precision, recall and F1-score can be reported. A confusion matrix is useful for identifying visually similar hand shapes. For live inference, average processing time and frames per second should be measured on the target laptop. For text-to-sign generation, qualitative review is more appropriate at the prototype stage because a small rule-based gloss module cannot be judged by classification accuracy alone.

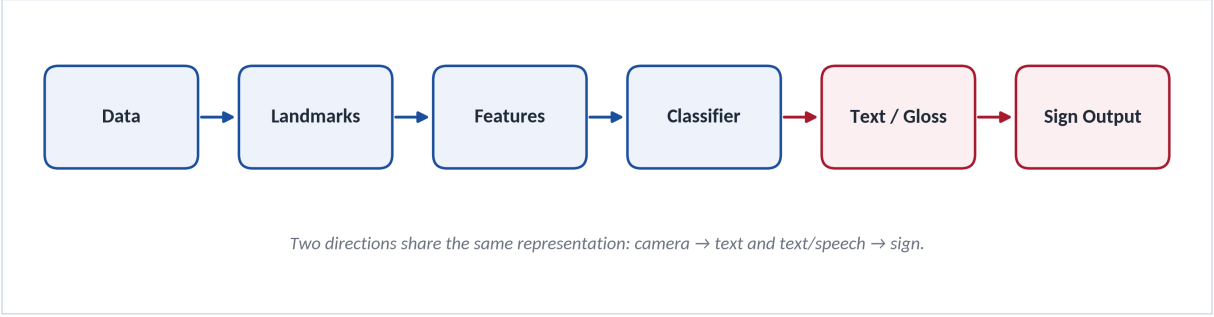


Figure 1: Proposed system workflow.

3.2. Users and Roles

Table 4: User Roles and Responsibilities

User / Role	Responsibility
End user	Provides gesture or text and reads translated output.
Developer	Collects samples, trains models and maintains the pipeline.
Evaluator	Checks recognition quality, usability and failure cases.
Dataset contributor	Provides labelled examples for vocabulary expansion.

3.3. Detailed Workflow

- Capture a webcam frame and detect the hand.
- Extract 21 landmarks and convert them to numerical features.
- Normalise coordinates using the same process as training.
- Classify a static sign or append the frame to a dynamic sequence buffer.
- Display the predicted label and confidence.
- Convert text into a simple ASL-oriented gloss sequence.
- Retrieve a matching sign sequence or fall back to fingerspelling.

3.4. Design Diagrams

3.4.1. Use-case Diagrams

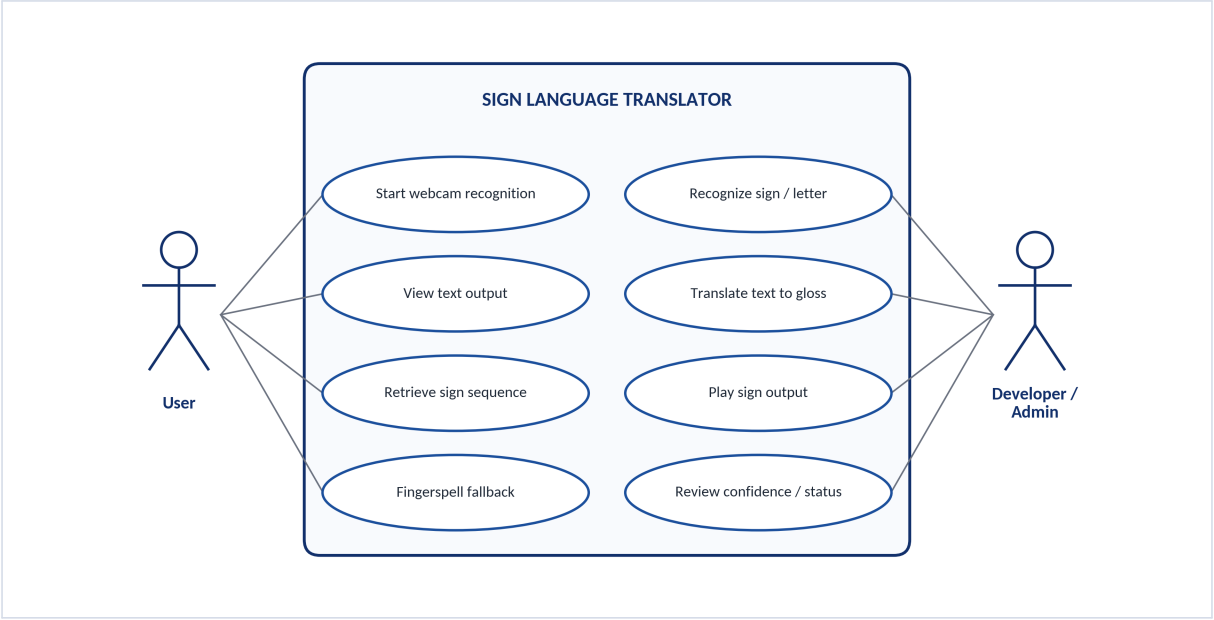


Figure 3: Use-case representation of the main user interactions.

3.4.2. Class Diagrams

The prototype can be organised into components rather than a large inheritance hierarchy: CameraInput, LandmarkExtractor, FeatureNormalizer, SignClassifier, SequenceBuffer, GlossConverter, SignRepository and OutputRenderer. This separation keeps computer vision independent from language processing and presentation.

3.4.3. ER / Data Architecture

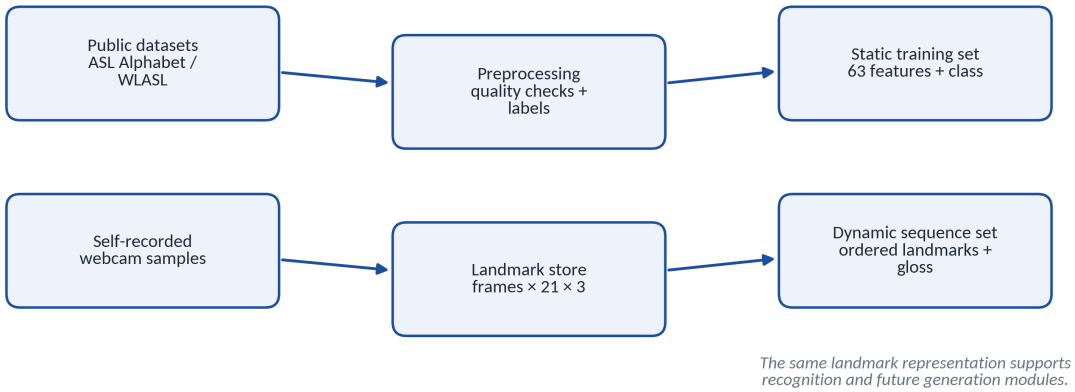


Figure 6: Data architecture for labelled landmarks, sign metadata and output mappings.

3.5. User Interface

3.5.1. User Description and Mockup

The interface is designed around two simple modes. In “Sign to Text”, the webcam view is shown with detected hand landmarks and the current prediction. In “Text to Sign”, the user enters a sentence, sees the intermediate gloss, and then sees the available sign sequence or fingerspelling output. Showing intermediate stages makes errors easier to understand during development.

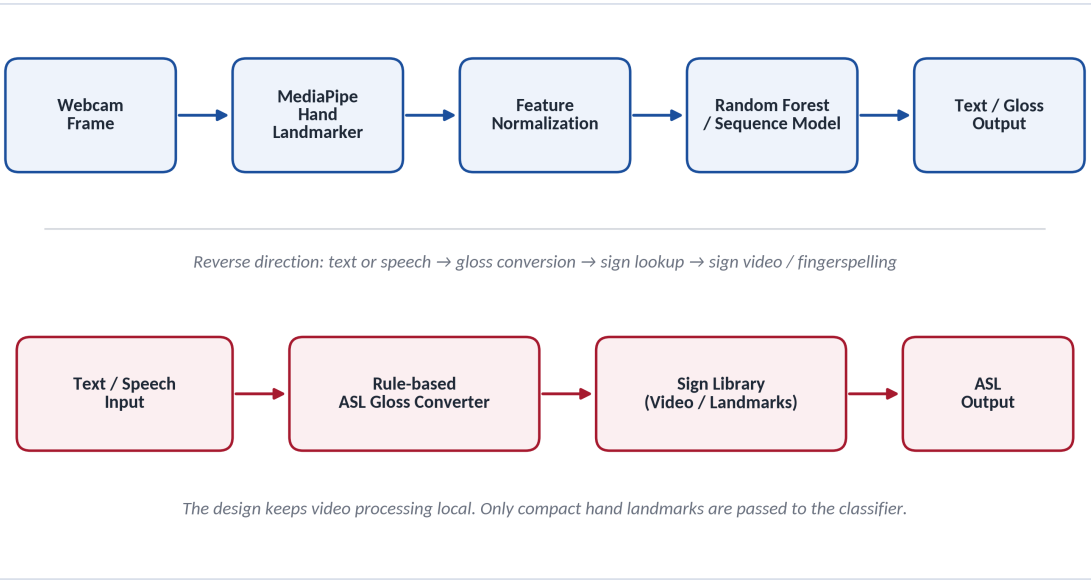


Figure 2: High-level system architecture.

3.6. System Implementation

The implementation is organised into independent modules: data collection, landmark extraction, preprocessing, model training, live recognition, text processing and sign playback. Python provides the main development environment because it has mature libraries for computer vision and machine learning. OpenCV is used for webcam capture and frame handling. MediaPipe provides hand landmark extraction. NumPy stores numerical feature vectors, while pandas can be used for labelled datasets. scikit-learn provides the Random Forest classifier and standard evaluation metrics. A lightweight interface can be built with Tkinter or a simple web-based front end depending on the final deployment choice.

Dataset preparation begins by assigning a label to each sample. Static samples are converted into one row per frame, containing the 63 landmark values and the target class. Training and testing data should be separated before model fitting. Where possible, samples from the same recording session should not be allowed to leak into both sets, because that can make the measured performance look better than real-world performance.

The live application follows the same preprocessing steps used during training. This consistency is essential: a classifier trained on normalised coordinates should receive the same normalisation at inference time. The predicted label is then displayed with a confidence indication. For dynamic signs, the application maintains a short rolling buffer of landmark frames and passes the sequence to the selected temporal model once sufficient training data are available.

The text-to-sign interface accepts typed text, performs the gloss conversion, and then resolves each gloss token against the sign library. The design allows recorded landmark sequences, sign videos or an avatar renderer to be used as the output layer without changing the input processing stage.

Table 3: System Resources and Tools

Resource	Purpose
Python 3.x	Main development language
OpenCV	Webcam capture and image handling
MediaPipe	Hand landmark detection
NumPy / pandas	Feature and dataset handling
scikit-learn	Random Forest training and evaluation
Local sign library	Stored sign sequences / videos
Laptop + webcam	Target development platform

3.6.1. ALGORITHMS / PSEUDO CODE

ALGORITHM: STATIC SIGN RECOGNITION

1. Start webcam capture.
2. Read the next frame.
3. Detect a hand using MediaPipe.
4. If no hand is detected, display “No hand detected”.
5. Extract 21 landmark coordinates.
6. Translate and scale coordinates for normalisation.
7. Flatten the 21 × 3 values into a 63-value feature vector.
8. Pass the vector to the trained Random Forest classifier.
9. Apply confidence thresholding or temporal smoothing.
10. Display the predicted sign label.
11. Repeat until the user exits.

3.6.2. SOURCE CODE WITH DOCUMENTATION

The source code should be maintained as separate modules for data collection, preprocessing, training, live inference and text-to-sign generation. A representative training flow is: load labelled landmark rows → split train/test data → fit RandomForestClassifier → evaluate on the held-out set → save the trained model → load the model in the webcam application. The final submission should include the project repository link and a README containing installation and execution steps.

GitHub repository: _____

4 CONCLUSION

4.1. Goals / Vision

The proposed mini-project demonstrates a practical route toward sign-language technology without requiring the full computational cost of a raw-video deep-learning system. By converting webcam frames into hand landmarks first, the project reduces the learning problem to compact geometric features and makes the recognition process easier to inspect and reproduce. The main delivered concept is a bidirectional pipeline. In the recognition direction, a webcam image is transformed into landmarks and classified into a sign label or letter. In the generation direction, text is converted into a simple gloss representation and mapped to available sign sequences, with fingerspelling available when a direct sign is missing. The architecture also preserves dynamic landmark sequences so that a stronger temporal model can be introduced later.

The current prototype should not be presented as a complete ASL translator. Natural sign-language translation involves facial expression, body movement, spatial reference, discourse context and language-specific grammar. The rule-based gloss stage is intentionally limited and transparent. Likewise, any numerical accuracy or latency figures should be reported only after measurements are collected on the final dataset and hardware.

Future development can therefore proceed in clear stages: enlarge the vocabulary, collect more signer-diverse data, replace static classification with sequence models, improve English-to-ASL linguistic processing, include non-manual features, and evaluate the complete system with signer-independent test sets. This staged approach keeps the project achievable while leaving a credible path toward a more capable system.

4.2. Delivered Solution

The intended deliverable is a working educational prototype that accepts webcam gestures and text, exposes hand landmarks and intermediate processing, performs static sign recognition, and provides a structured route to sign output. The design prioritises reproducibility and clear separation of components.

4.3. Remaining Work

- Collect a larger, signer-independent dataset and report measured test metrics.
- Add a sequence model for dynamic signs.
- Improve English-to-ASL linguistic processing beyond simple rules.
- Include facial and body features where required by the target signs.
- Test usability with a broader range of users and recording conditions.

5 REFERENCES

[1] D. Li, C. Rodriguez, X. Yu and H. Li, "Word-level Deep Sign Language Recognition from Video: A New Large-scale Dataset and Methods Comparison," *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pp. 1459–1469, 2020.

[2] H. Vaezi Joze and O. Koller, "MS-ASL: A Large-Scale Data Set and Benchmark for Understanding American Sign Language," *British Machine Vision Conference (BMVC)*, 2019.

[3] A. Desai et al., "ASL Citizen: A Community-Sourced Dataset for Advancing Isolated Sign Language Recognition," *arXiv:2304.05934*, 2023.

[4] N. C. Camgöz, S. Hadfield, O. Koller, H. Ney and R. Bowden, "Neural Sign Language Translation," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[5] F. Zhang et al., "MediaPipe Hands: On-device Real-time Hand Tracking," *CV4ARVR*, 2020.

[6] J. M. Koller, "Sign Language Recognition and Translation: A Review," and related survey literature on sign-language recognition and translation; used here as background for distinguishing recognition, translation and production tasks.

[7] D. Li et al., "Transferring Cross-Domain Knowledge for Video Sign Language Recognition," *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6205–6214, 2020.

[8] O. Koller, J. Forster and H. Ney, "Continuous Sign Language Recognition: Towards Large Vocabulary Statistical Recognition Systems Handling Multiple Signers," *Computer Vision and Image Understanding*, vol. 141, pp. 108–125, 2015.